Docker Level 1 — Tasks 1 to 5

Hands-on runbook • Syed Aftab • Ubuntu (WSL) • install, run, delete, copy, troubleshoot • Reference for real work & interviews

What this covers: the five KodeKloud Docker Level 1 tasks, done for free on your own machine: verify an install, deploy nginx, delete a container, copy a file in, and troubleshoot a container that crashes after a bad config change.

The big ideas

Idea	Meaning
image vs container	an **image** is the template (nginx:alpine); a **container** is a running (or stopped) copy of it. Removing a container keeps the image
client vs daemon	`docker` is the **client** you type; `dockerd` is the **daemon** (service) that does the work, passing containers to containerd and runc
own filesystem	a container has its own folders — it can't see your host's files unless you copy or mount them
own processes	inside, the app is **PID 1**, with no systemd. **When PID 1 exits, the container stops**
docker logs	the container's version of `journalctl -u`: everything PID 1 printed, even after it stopped
docker cp changes	live only in that container — gone when it's removed. Real setups build files into the image or mount them

TASK 1 Install Docker and start the service

The state we found: Docker was already installed (Ubuntu's `docker.io` package, Git commit `29.1.3-0ubuntu4.1`) and running. Instead of reinstalling, we did the task as **verify and understand**.

```
systemctl status docker --no-pager
   Loaded: ... docker.service; enabled
   Active: active (running) since Fri 2026-10-30 17:06:26 EDT; 3 days ago
TriggeredBy: * docker.socket
 Main PID: 432 (dockerd)
   CGroup: ... 432 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

Clue	Meaning
`TriggeredBy: docker.socket`	socket activation (like SSH): systemd listens for Docker requests and can start the engine on demand
`dockerd`	the Docker engine (daemon). `docker` is only the client
`-H fd://`	listen on the socket systemd hands over
`--containerd=...`	dockerd passes the actual running of containers to containerd
"use -l to show in full"	log lines are cut off; `-l` shows them whole

The install commands (reference):

Distro	Install	Start + enable
Ubuntu	`sudo apt install -y docker.io docker-compose-v2`	`sudo systemctl enable --now docker`
RHEL / CentOS / Rocky	`sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin` (after adding Docker's repo)	`sudo systemctl enable --now docker`

Part	Meaning
`docker.io` / `docker-ce`	Ubuntu's package vs Docker Inc's official Community Edition — same behaviour
`docker-compose-v2` / `-plugin`	adds `docker compose` (several containers from one YAML file)
remove (keep data)	`sudo systemctl stop docker docker.socket containerd` then `sudo apt remove -y docker.io ...`. `remove` keeps `/var/lib/docker` (images, containers, volumes); `purge` also deletes config

Verify — the way a task checker would:

```
systemctl is-active docker        # active
systemctl is-enabled docker       # enabled
docker compose version            # Docker Compose version v5.5.0
sudo docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:c567b231f656...
Hello from Docker!
```

Part / output	Meaning
`is-active` / `is-enabled`	one-word answers: running now? starts at boot?
`docker run --rm hello-world`	end-to-end test: client → daemon → pull from Docker Hub → run → print. `--rm` deletes the container on exit
`hello-world:latest`	`latest` = the **tag** (version). `library/` = Docker's official images
`4f55086f7dd0: Pull complete`	images are made of **layers**, each downloaded separately
`Digest: sha256:...`	the image's unique fingerprint
no sudo needed	your user is in the **docker** group — effectively **root-level access** (containers can mount host files)

TASK 2 Deploy an nginx container

```
docker run -d --name nginx_1 -p 8080:80 nginx:alpine
docker ps
CONTAINER ID  IMAGE         ...  STATUS     PORTS                                     NAMES
9ee78dda9d07  nginx:alpine  ...  Up 8 min   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   nginx_1
```

Part	Meaning
`docker run`	create **and** start a new container from an image
`-d`	**d**etached: run in the background, give the prompt back
`--name nginx_1`	fixed name (otherwise random like happy_turing). Checkers look for the exact name
`-p 8080:80`	**p**ublish a port, HOST:CONTAINER — host 8080 → nginx's 80 inside
`nginx:alpine`	image nginx, tag alpine (tiny ~5 MB Linux base)
`docker ps`	running containers: ID, image, status, ports, name
`0.0.0.0:8080->80/tcp`	every IPv4 address, host port 8080, forwarded to container port 80, TCP

Verified in the Windows browser at localhost:8080 ("Welcome to nginx!"): browser → WSL → port mapping → nginx in the container.

Common mistake: forgetting `-p`. nginx still runs, but only Docker's internal network can reach it — "works but can't be reached".

TASK 3 Delete a container

```
docker stop nginx_1
docker ps            # gone from running list
docker ps -a         # still listed: Exited (0)
docker rm nginx_1
docker ps -a         # gone completely
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS   NAMES
```

Command	Meaning
`docker stop`	graceful stop: SIGTERM, wait 10 s, then SIGKILL (polite first, like kill in Scenario #3)
`docker ps -a`	**a**ll containers, including stopped
`docker rm`	re**m**ove a stopped container for good
`docker rm -f`	**f**orce: stop and remove in one step (like kill -9 — fine for test containers)
`docker rmi IMAGE`	remove an **image** (rm never touches images)
`docker ps 0a`	typo → "docker ps accepts no arguments" — options need the dash: `-a`

State	docker ps	docker ps -a	Start again?
running	yes	yes	already running
stopped (exited)	no	yes	`docker start NAME`
removed	no	no	no — `docker run` a new one

TASK 4 Copy a file into a container

```
docker run -d --name nginx_1 -p 8080:80 nginx:alpine
echo "<h1>Deployed by Syed with docker cp</h1>" > index.html
bash: index.html: Permission denied            <-- we were in /home, then /
sudo echo "..." > index.html
bash: index.html: Permission denied            <-- sudo doesn't cover the >
```

Problem	Why	Fix
Permission denied in /home and /	both owned by root, `drwxr-xr-x` — creating a file needs **w on the folder** (Scenario #5)	`cd ~` — work in your own home
`sudo echo ... > file` still denied	`sudo` applies only to echo; the > is done by **your** shell, as you, first	for root-owned files: `echo ... | sudo tee FILE`

```
cd ~
echo "<h1>Deployed by Syed with docker cp</h1>" > index.html
ls -l index.html            # -rw-r--r-- 1 syedaftab04 syedaftab04 41 ... index.html
docker cp index.html nginx_1:/usr/share/nginx/html/index.html
docker exec nginx_1 cat /usr/share/nginx/html/index.html
<h1>Deployed by Syed with docker cp</h1>
curl -I http://localhost:8080      # 200 OK, Content-Length: 41
```

Part	Meaning
`docker cp SRC CONTAINER:PATH`	copy host → container (and the reverse: `docker cp nginx_1:/etc/nginx/nginx.conf .`)
`nginx_1:`	which container; the colon separates name from path (like scp)
`/usr/share/nginx/html/`	nginx's web root inside the container; index.html is the page at /
`docker exec CONTAINER CMD`	run a command **inside** a running container — proves the file arrived
`Content-Length: 41`	exactly the 41-byte file you created — nginx is serving your copy

Check	Proves
`docker cp`	the copy ran
`docker exec ... cat`	the file is **inside** the container
`curl`	the app is actually **serving** it

Getting inside a container

```
docker exec -it nginx_1 sh
/usr/share/nginx/html # ls
50x.html    index.html
/usr/share/nginx/html # ps
PID   USER   COMMAND
  1   root   nginx: master process nginx -g daemon off;
 30   nginx  nginx: worker process      (x6)
/ # exit
```

Part / output	Meaning
`-i` / `-t`	**i**nteractive input / a **t**erminal — together, a usable shell
`sh` vs `bash`	Alpine images only have sh; Ubuntu/Debian images have bash
# prompt	you are root inside the container
`50x.html`	nginx's default error page (for 502/503)
PID 1 = nginx	no systemd inside. Own PID **namespace**: it can't see host processes
`-g daemon off;`	keep nginx in the **foreground** — if PID 1 exits, the container stops
master root, workers nginx	same design as a server: master binds port 80, unprivileged workers serve requests
`exit`	leaves your shell only; the container keeps running
`docker exec -it -u root ...` / `docker run -it --rm ubuntu bash`	go in as root / a throwaway Ubuntu container
exec on a stopped container	fails ("is not running") — investigate from outside: ps -a, logs, inspect

Catch: a file added with `docker cp` lives only in that container. `docker rm` + a new `docker run` = back to the default page. Real setups put files in the **image** (Dockerfile) or **mount** them.

TASK 5 Troubleshoot a container that won't stay up

The deployment (setup): the team's config, copied into a new container, then a restart.

```
nano ~/default.conf
server {
    listen 80
    root /usr/share/nginx/html;
}
docker run -d --name web_app -p 8081:80 nginx:alpine
docker cp ~/default.conf web_app:/etc/nginx/conf.d/default.conf
docker restart web_app
```

Part	Meaning
`server { ... }`	one website (server block)
`listen 80` / `root ...;`	listen on port 80 inside / serve pages from this folder
`/etc/nginx/conf.d/default.conf`	nginx's site config inside the container
`docker restart`	stop + start. nginx reads config at startup, so a copied config takes effect on restart
`Successfully copied 2.05kB`	the size of the package docker cp sends (includes overhead), not the file

Confirm and locate

```
curl -I http://localhost:8081
curl: (7) Failed to connect to localhost port 8081 ... Could not connect to server
docker ps -a
CONTAINER ID  IMAGE         STATUS                      PORTS                  NAMES
03f46160912c  nginx:alpine  Exited (1) 5 seconds ago                           web_app
9ee78dda9d07  nginx:alpine  Up 24 minutes               0.0.0.0:8080->80/tcp   nginx_1
```

Clue	Meaning
curl error 7	nothing listening on 8081
`docker ps` without web_app	not running — crashed or never created? `docker ps -a` answers that
`Exited (1)`	it **crashed**; 1 = error exit code (like status=1/FAILURE in #5)
empty PORTS	ports only show while running — that's why 8081 refuses
nginx_1 Up	Docker itself is healthy; the problem is specific to web_app

Identify — docker logs

```
docker logs web_app
/docker-entrypoint.sh: ... Configuration complete; ready for start up
2026/11/03 13:05:06 [emerg] 1#1: invalid parameter "root" in /etc/nginx/conf.d/default.conf:3
nginx: [emerg] invalid parameter "root" in /etc/nginx/conf.d/default.conf:3
```

Part / line	Meaning
`docker logs NAME`	everything PID 1 printed — **works after the container stopped**. `--tail 20` / `-f` like journalctl
`/docker-entrypoint.sh ...` lines	the image's startup script preparing things — normal
`[emerg]`	nginx refuses to start
`1#1`	PID 1 = nginx. It exited → the container stopped
`invalid parameter "root" ... :3`	nginx choked on line 3 — but **line 2** `listen 80` is missing its `;` (the Scenario #2 rule: check the line above)

Fix and verify

```
nano ~/default.conf                       # line 2 -> listen 80;
docker cp ~/default.conf web_app:/etc/nginx/conf.d/default.conf
docker start web_app
docker ps -a
03f46160912c  nginx:alpine  Up 3 seconds  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp  web_app
```

Part	Meaning
`docker cp` to a **stopped** container	works — unlike docker exec. Fix files without the container running
`docker start NAME`	start the existing stopped container (same name, ports, files). `docker run` would make a new one
`Up 3 seconds` + ports back	fixed

Ticket summary: a config missing a ';' on line 2 was copied into web_app and the container restarted. nginx (PID 1) failed its config check and exited, so the container stopped and port 8081 refused connections. Fixed the file, copied it back, started the container.

Prevention — test before restarting (same as nginx -t && reload in Scenario #2):

```
docker cp ~/default.conf web_app:/etc/nginx/conf.d/default.conf
docker exec web_app nginx -t && docker exec web_app nginx -s reload
```

Part	Meaning
`docker exec web_app nginx -t`	test the new config inside the running container
`&&`	only continue if the test passed
`nginx -s reload`	apply it without restarting the container — a bad config never takes the site down

Cleanup

```
docker rm -f nginx_1 web_app
```

REF Command reference

Command	What
`systemctl is-active / is-enabled docker`	service running? starts at boot?
`docker version` / `docker compose version`	client + engine versions / compose plugin
`docker run --rm hello-world`	end-to-end install test
`docker run -d --name N -p H:C IMAGE:TAG`	create + start a container in the background
`docker ps` / `docker ps -a`	running / all containers
`docker stop`, `start`, `restart NAME`	stop gracefully / start existing / both
`docker rm NAME` / `rm -f`	remove stopped / force stop + remove
`docker rmi IMAGE`	remove an image
`docker cp SRC NAME:PATH` (and reverse)	copy files host ↔ container (works on stopped containers)
`docker exec NAME CMD`	run a command inside a running container
`docker exec -it NAME sh`	a shell inside the container
`docker logs NAME` (`--tail N`, `-f`)	the container's output — works after it stopped
`docker inspect NAME`	full details as JSON (state, exit code, mounts, IP, ports)

Interview answer: "a container keeps exiting"

"docker ps doesn't show it, so docker ps -a: Exited (1), an error exit. Inside a container the app is PID 1, so the container stops when the app exits — the question is why the app exited. I can't exec into a stopped container, so I read docker logs: nginx reported an invalid parameter on line 3 of its config, which really meant a missing semicolon on line 2. I fixed the file, copied it back with docker cp — that works on stopped containers — and started it again. To prevent it, test the config with docker exec nginx -t before reloading, and keep config in the image or a mounted file rather than docker cp, so it's versioned and survives a rebuild."